# CSE470: Software Engineering

## Project Outline and Development Guidelines

**Project Goal**

Students will work in teams to analyze, design, implement, test, document, and demonstrate a software project using a framework that supports the MVC (Model, View, Controller) architecture. The project is intended to provide practical experience in software engineering practices, collaborative development, version control, iterative delivery, and technical communication.

## 1. Learning Objectives

- Translate a real-world problem into clear software requirements and well-defined features.
- Design and implement a software system using MVC with clear separation of responsibilities.
- Apply software engineering practices including Git-based version control, documentation, iterative development, testing, debugging, and code review.
- Work effectively as a team using a one-week sprint cycle with explicit weekly goals and attainment reporting.
- Explain and defend individual technical decisions and contributions during the project viva.
- Use development tools, including AI-assisted tools where appropriate, responsibly and with sufficient understanding of all submitted work.

## 2. Project Format and Team Structure

- One project will be developed by a group of four (4) students
- Every member must make a meaningful technical contribution. Team membership does not transfer individual responsibility for understanding the project.
- Each member must be able to explain the architecture, relevant requirements, implementation choices, Git history, and their own contributions during the viva.
- The project idea must be sufficiently complex for a fourth-year Software Engineering course. The section faculty may approve, modify, reject, or merge project ideas to maintain an appropriate level of complexity and avoid excessive duplication.

# 3. Requirement and Feature Definitions

The project must distinguish between a requirement and a feature. This distinction is important because individual CRUD actions or interface buttons should not automatically be counted as separate features.

| Term | Definition | Example |
|---|---|---|
| Requirement | A user-facing capability or system behavior that the software must provide to satisfy a meaningful need. | Customers can manage products in their cart. |
| Feature | A coherent, testable functional unit that contributes to fulfilling a requirement. A feature normally involves a meaningful user/system workflow rather than a single database or UI operation. | Cart management: view cart, add an available product, update quantity, remove an item, and calculate the cart total. |

**Feature Counting Rule**

A feature must represent a meaningful software capability or workflow that can be demonstrated and tested. Simple CRUD operations, individual buttons, individual form fields, page navigation, login/registration, logout, password reset, and similar basic supporting actions do NOT count as separate features unless the faculty explicitly approves them as a substantial standalone capability.

## 3.1 What Counts as One Feature?

- A feature should have a clear purpose, user/system interaction, expected outcome, and acceptance criteria.
- A feature may contain multiple related actions. For example, 'Product Management' can include creating, viewing, editing, deleting, searching, filtering, and validating products when these actions form one coherent management capability.
- A feature may be decomposed into implementation tasks for a sprint, but those tasks do not automatically become separate project features.
- The same functionality implemented for different user roles is not automatically multiple features. The distinction must be justified by materially different workflows or capabilities.
- Students must identify at least 20 substantive features for the complete project. The section faculty has final authority over whether the submitted feature list meets the required complexity.

### 3.2 Examples

| Classification | Example |
|---|---|
| Not acceptable as separate features | Add product, edit product, delete product, product form, product list, product details page. |
| Acceptable feature | Product Management: an integrated capability allowing an authorized user to create, view, update, delete, search, and manage product information. |
| Not acceptable as features | Login, logout, registration, forgot password. |
| Acceptable feature | Order Tracking: showing active orders, maintaining status transitions, storing relevant order information, and presenting current order status to the customer. |

## 4. Project Scope and Complexity

- The project should solve a clearly defined problem for a specific target user or organization.
- The project should have a reasonable combination of frontend, backend, database, business logic, and validation rather than being primarily a static interface.
- The project should contain at least 20 approved substantive features.
- The project should include multiple related entities and meaningful business rules where appropriate to the problem domain.
- Students should avoid projects that are only a basic blog, simple to-do list, simple CRUD dashboard, static information website, or another application whose complexity is too low for a fourth-year course.
- The faculty may require additional features, reduce scope, or change the project if the proposed system is too simple, too large for the schedule, technically unsuitable, or substantially duplicates another team's project.

## 5. Technology Stack and Framework Requirements

The project must use a framework that supports and visibly demonstrates MVC architecture. Students are responsible for understanding the framework's architecture rather than merely using a framework name in the project report.

- Students may choose a web, mobile, desktop, or other application type only when the selected framework and implementation can clearly demonstrate MVC separation.
- Examples of suitable MVC-oriented technologies include Laravel, ASP.NET MVC/Core MVC, Spring MVC, Ruby on Rails, and other faculty-approved MVC frameworks.
- JavaScript-based frameworks may be used where the overall project architecture remains consistent with the course's MVC requirement. The student must clearly identify the Model, View, and Controller responsibilities in the project. However, you cannot use MERN Stack or any combination of this stack.
- The primary application/backend must NOT be implemented using a Python framework. Python frameworks such as Django, Flask, FastAPI, Streamlit, or similar frameworks are not permitted for the main project.

Exception: Python may be used as a separate AI/ML backend or service when the project contains a genuine AI/ML feature that requires Python-based tooling. In such cases, the main application must still use an approved MVC framework, and the Python component must be limited to the AI/ML functionality.

- The AI/ML service must communicate with the main application through a clearly defined interface/API. Students must be able to explain the data flow between the MVC application and the AI component.
- Using a technology simply because it is familiar or easier is not sufficient justification. Students should select a stack that is appropriate for the project's requirements and can be completed reliably within the project schedule.

### 5.1 Recommended Stack Selection Documentation

The project proposal must include:

- Programming language(s).
- Primary framework and version.
- Frontend technology/framework, if separate.
- Database and database technology.
- External APIs/services, if any.
- AI/ML framework or service, if applicable.
- Reason for selecting the stack and framework.
- A brief explanation of how MVC will be implemented.

## 6. MVC Architecture Requirements

- The project MUST follow MVC architecture throughout the application.
- Models should represent and manage application data and relevant data logic.
- Views should be responsible for presentation and user interaction rather than containing substantial business logic.
- Controllers should coordinate requests, application flow, validation, and communication between views and models as appropriate.
- Students must avoid placing substantial business logic directly in views or creating controllers that contain all application logic.
- The project should demonstrate reasonable separation of concerns, modularity, reusable components, and maintainable code.
- The viva may include questions requiring students to identify specific Models, Views, and Controllers and explain why code belongs in each layer.

## 7. Libraries, Packages, APIs, and External Components

- Students may use standard libraries, framework packages, UI libraries, utility packages, and external APIs when they support development without implementing a major assessed feature on the students' behalf.
- A package that directly provides a major project capability may not be used as a substitute for implementing that capability unless explicitly approved by the faculty.
- Examples that require faculty approval include packages that provide an entire core workflow, complete business module, major recommendation engine, or substantial assessed functionality with minimal student implementation.
- Students must record significant third-party packages and APIs in the project documentation and explain their purpose.
- Students must comply with the licenses and usage requirements of external libraries, APIs, datasets, and services.
- When uncertain whether a package is permitted, students must consult the section faculty before using it.

## 8. AI-Assisted Development Policy

**Core Principle**

AI tools may be used as development assistants, but the project must be designed, implemented, debugged, and understood by the students. Students must NOT use AI to generate the entire project and submit the generated code as their own work.

- AI tools may be used for learning concepts, brainstorming, documentation assistance, debugging suggestions, code explanations, refactoring suggestions, test-case ideas, and other reasonable development assistance.
- Students remain responsible for verifying, modifying, integrating, testing, and understanding any AI-assisted output used in the project.
- Students must not prompt an AI system to generate the entire project, entire modules, or a complete feature set and then submit the output without meaningful student implementation and understanding.
- Students must not use AI to fabricate requirements, test results, Git history, sprint evidence, citations, or other project records.
- If AI-generated code is used, the student must be able to explain the code, its design decisions, its dependencies, its limitations, and how it integrates with the rest of the application.
- The faculty may ask a student to modify, extend, debug, or rewrite an AI-assisted section during the viva. Inability to explain or reproduce the logic may negatively affect the viva assessment.
- AI use does not reduce individual responsibility. The 10-mark viva specifically assesses whether each student genuinely understands the project and their contribution.

### 8.1 Disclosure of AI Use

The final submission should contain a short AI-use statement identifying:

- Which AI tools were used, if any.
- The purpose of using them (e.g., debugging explanation, documentation assistance, test-case suggestions).
- Which parts of the project were materially influenced by AI assistance.
- A statement confirming that the submitted work was reviewed, tested, and understood by the students.

## 9. Project Proposal and SRS

Before Sprint 1, each team must submit an approved project proposal/SRS containing at minimum:

- Project title and problem statement.
- Target users and system goals.
- Functional requirements.
- At least 20 proposed substantive features.
- Non-functional requirements such as performance, usability, security, reliability, or maintainability where relevant. (Optional)
- Feature descriptions and acceptance criteria.
- User roles and permissions.
- High-level system architecture and MVC mapping.
- Database/entity overview.
- Selected technology stack and justification.
- Team member responsibilities.
- Initial sprint plan of 4 weeks
- Known external APIs, packages, datasets, or AI components.

## 10. One-Week Sprint Process

The project will use a one-week sprint cycle. The standard project period consists of four (4) weekly sprints unless the section faculty specifies otherwise.

- Sprint 1, Sprint 2, Sprint 3, and Sprint 4 will each have a duration of approximately one week.
- At the start of each week(Saturday), the team must communicate its sprint goals and planned work.
- At the end of each week(Friday), the team must report what was completed, what was partially completed, what was not completed, and why.
- Sprint goals must be specific and measurable. 'Work on backend' is not an adequate sprint goal; 'Implement and test product search with category and price filters' is a suitable example.
- Incomplete work must not be hidden. Students are expected to report unfinished tasks, bugs, blockers, and scope changes honestly.
- Sprint scope may be adjusted based on the previous sprint, but changes should be documented.

- Each sprint should result in a usable increment of the project where practical.

### 10.1 Sprint Review Ceremony

Each weekly sprint will include the following two checkpoints:

| Checkpoint | What students submit/share | Purpose |
|---|---|---|
| Sprint Start (Saturday) | Weekly goals, assigned tasks, target features, expected deliverables, blockers if already known. | Establish a clear and measurable plan for the week. |
| Sprint End / Review (Friday) | Completed features/tasks, Git activity, demo/evidence where requested, unfinished work, blockers, lessons learned, next steps. | Evaluate attainment against the planned goals and maintain accountability. |

## 11. Version Control and Collaboration

- GitHub is mandatory for version control.
- The section faculty must be added as a collaborator to the repository during Sprint 1.
- Each student must make meaningful commits throughout the project. A single end-of-semester bulk upload is not acceptable evidence of continuous development.
- Students should use branches, pull requests, issues, or other appropriate GitHub practices when useful for collaboration.
- Each student must push their current work at the end of every sprint, including incomplete work, known bugs, and partially implemented features.
- Commit messages should communicate what was changed. Students should avoid meaningless messages such as 'update', 'final', or 'done'.
- Git history may be considered as evidence of individual contribution and sprint progress.

## 12. Testing and Quality Requirements

- Each major feature should have clear expected behavior and test scenarios.
- Students should test normal, invalid, and boundary cases where applicable.
- The project should include reasonable validation and error handling.
- Known bugs and limitations should be documented rather than concealed.
- Students should perform integration testing for important workflows involving multiple components.
- Security-related concerns such as authorization, input validation, password handling, and protection of sensitive information should be considered where applicable.

## 13. Final Submission Requirements

- Complete source code in the approved GitHub repository.
- Recorded single video of the features where each member will explain their part
- Final SRS/project documentation.
- Feature list with completion status.
- Sprint records showing weekly goals and attainment.
- README containing setup instructions, technology stack, environment requirements, and how to run the application.
- AI-use disclosure, if AI tools were used.
- List of significant third-party packages, APIs, datasets, or services.
- Any additional materials requested by the section faculty.

## 14. Project Viva

The project viva is an individual assessment. The project team may demonstrate the application together, but each student is assessed individually.

- Each student will attend a 10-minute viva.
- Approximately 5 minutes will be used for project demonstration.
- Approximately 5 minutes will be used for individual questions about implementation, architecture, code, requirements, Git activity, and the student's contribution.
- Questions may cover parts of the project beyond the student's self-declared tasks because every team member is expected to understand the overall system.
- Students may be asked to explain or modify code, identify MVC components, trace a request through the system, justify a design choice, or debug an issue.
- The viva may be conducted online or offline at the discretion of the section faculty.
- Vivas may be recorded for academic review where feasible.

**Important**

Because the viva carries 10 out of 15 marks, students should treat understanding and ownership of the implementation as a core project requirement. Using tools, libraries, or AI systems without understanding the resulting code can directly affect the student's ability to demonstrate competency.

## 15. Marking Rubric

| Component | Marks | What is assessed | Key evidence |
|---|---|---|---|
| SRS + Sprint Process and Version Control | 2 | Clarity, completeness, quality of requirements and feature definitions, architecture documentation. Weekly planning, attainment reporting, | SRS, feature list, diagrams, documentation, Sprint submissions, Git history, issues/branches/PRs where applicable |
| Implementation / MVC / Project Quality | 3 | Functional implementation, MVC adherence, code quality, maintainability, validation, testing and overall quality. | Working application, source code, architecture, tests |
| Project Viva (Individual) | 10 | Individual contribution, technical understanding, code explanation, architecture, debugging/problem solving and ability to defend decisions. | Demo, Q&A, code walkthrough |

**Total: 15 marks**

## 16. Academic Integrity and Ownership

- Students must submit work that they can legitimately explain and defend.
- Copying code, using another person's project/codebase, submitting a previous project, or misrepresenting another person's work as the team's own is prohibited.
- Students may consult official framework documentation and general learning resources, but they must not copy a tutorial project or codebase as the project implementation.
- Third-party code, datasets, images, APIs, and libraries must be used according to their licenses and disclosed where required.
- Fabricating Git activity, sprint progress, test results, or individual contributions is prohibited.
- Academic-integrity concerns may be handled according to university/course policy.

## 17. Student Checklist

- ☐ Our project solves a clearly defined problem for identified users.
- ☐ Our project has at least 20 substantive, faculty-approved features.
- ☐ We have not counted simple CRUD actions or basic authentication actions as separate features.
- ☐ Our main application uses an approved MVC-oriented framework.
- ☐ We are not using a Python framework for the main application.
- ☐ Any Python component is limited to a genuine AI/ML backend feature and is connected to the MVC application through a defined interface.

- ☐ We have submitted an SRS and documented our stack and MVC design.
- ☐ We have a one-week sprint plan and will report goals at sprint start and attainment at sprint end.
- ☐ Every team member has meaningful Git activity and understands the overall project.
- ☐ We have tested our major features and documented important limitations.
- ☐ We have used AI, if at all, as an assisting tool rather than as a substitute for implementation and understanding.
- ☐ Every team member can explain and defend the submitted code during the viva.

## 18. Faculty Approval and Final Authority

This outline establishes the minimum project expectations. The section faculty may provide additional requirements, templates, deadlines, approval criteria, or technical restrictions when necessary for the section. Where a project idea, feature, technology, or implementation approach is ambiguous, students must seek faculty approval before proceeding.